University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

# Fehlertoleranz und deterministische Zustandsrekonstruktion in MQTT-basierten IIoT- und MES-Architekturen

State of the Art, Fehlermodelle und Architekturvergleich

Exposé und Antrag auf Durchführung einer Masterarbeit
in Kooperation mit der [Firma]
Master Studiengang Informatik

**Autor:** Erlind Sejdiu
**Matrikelnummer:** 106357

**Betreuer Hochschule:** [Name]
**Betreuer Unternehmen:** [Name, Abteilung]
**Geplanter Zeitraum:** [TT.MM.JJJJ] – [TT.MM.JJJJ] (6 Monate)

Stand: 27. Oktober 2026

# Inhaltsverzeichnis


**1 Motivation** **1**

**2 Problemstellung** **1**

2.1 Was MQTT garantiert und was nicht . . . . . 1

2.2 Was zustandsbehaftete Prozesse brauchen . . . . . 1

2.3 Die Lücke . . . . . 2

**3 Forschungsfragen und Ziele** **2**

**4 Vorgehen** **2**

4.1 State of the Art und Related Work . . . . . 2

4.2 Fehlermodell . . . . . 3

4.3 Architekturvergleich . . . . . 3

4.4 Bewertung . . . . . 3

**5 Nutzen für die [Firma]** **4**

**6 Rahmenbedingungen** **4**

**7 Zeitplan** **4**

**Literaturverzeichnis** **5**

# 1 Motivation

Die industrielle Prozessdatenerfassung über Message Queuing Telemetry Transport (MQTT) bildet das nachrichtentechnische Rückgrat moderner Architekturen für das Industrial Internet of Things (IIoT), Fertigungsmanagementsysteme (Manufacturing Execution Systems, MES) sowie Unified Namespaces (UNS) [1]. Maschinen, Steuerungen und Edge-Gateways veröffentlichen ihre Daten auf einem zentralen Broker, und MES, Historian oder Dashboards abonnieren genau die Daten, die sie brauchen. Diese lose Kopplung ist der Grund für die Verbreitung von MQTT, denn Erzeuger und Verbraucher müssen sich nicht kennen [2].

Ein MES arbeitet aber nicht mit einzelnen Messwerten, sondern mit Zuständen. Es muss wissen, welcher Auftrag gerade auf welcher Maschine läuft, wie viele Gutteile gefertigt wurden und ob eine Maschine steht oder produziert [3]. Geht eine Nachricht verloren, kommt sie doppelt oder in falscher Reihenfolge an, dann ist dieser Zustand falsch. Ein verpasstes „Auftrag beendet“ oder ein doppelt gezählter Stückzähler wirkt sich direkt auf Kennzahlen, Rückmeldungen an das ERP-System und Nachweise gegenüber Kunden aus.

Die wissenschaftliche Literatur offenbart eine signifikante Diskrepanz zwischen den elementaren Transportgarantien des MQTT-Standards und den deterministischen Anforderungen zustandsbehafteter Automatisierungsprozesse. MQTT sichert die Zustellung nur zwischen einem Client und dem Broker ab, nicht von Ende zu Ende [4]. Was nach einem Broker-Neustart, einem Netzwerkausfall oder dem Neustart eines Verbrauchers mit dem Zustand passiert, regelt der Standard nicht. Erweiterungen wie Sparkplug B [5] und Muster aus verteilten Systemen wie Event Sourcing [6] oder Log-basierte Plattformen [7] schließen einen Teil dieser Lücke, jeweils mit eigenen Annahmen und Kosten.

Bei der [Firma] ist MQTT [kurz beschreiben, wo und wie MQTT im Unternehmen eingesetzt wird, z. B. Anbindung von Maschinen an das MES, UNS-Aufbau]. Die Masterarbeit soll klären, wie zuverlässig solche Architekturen den Zustand der Fertigung nach Fehlern wiederherstellen können und welche Architektur dafür am besten geeignet ist.

# 2 Problemstellung

## 2.1 Was MQTT garantiert und was nicht

MQTT bietet drei Dienstgüten (Quality of Service, QoS) [4], [8]. QoS 0 stellt höchstens einmal zu, QoS 1 mindestens einmal und QoS 2 genau einmal. Diese Garantien gelten aber jeweils nur für eine Strecke, also vom Publisher zum Broker und getrennt davon vom Broker zum Subscriber. Ob eine Nachricht beim Verbraucher tatsächlich verarbeitet wurde, ist damit nicht gesichert. Die Reihenfolge wird nur innerhalb eines Topics und einer Dienstgüte garantiert, bei Shared Subscriptions in MQTT 5 auch das nicht.

Für den Wiederanlauf gibt es zwei Mechanismen. *Retained Messages* speichern pro Topic genau die letzte Nachricht, und *persistente Sessions* puffern Nachrichten für einen getrennten Client. Beide sind kein Verlauf. Wer nach einem Ausfall wissen will, was in der Zwischenzeit passiert ist, bekommt bestenfalls den letzten Wert, und auch das nur, solange Puffergrenzen und Ablaufzeiten des Brokers nicht überschritten sind. Wie ein Broker seine Daten persistiert und ob er im Cluster betrieben werden kann, ist nicht standardisiert, sondern produktspezifisch.

## 2.2 Was zustandsbehaftete Prozesse brauchen

Ein MES leitet seinen Zustand aus einer Folge von Ereignissen ab, zum Beispiel Auftragsstart, Statuswechsel und Stückmeldungen. Damit dieser Zustand nach einem Fehler korrekt wiederhergestellt werden kann, muss er *deterministisch* aus den Ereignissen folgen, also bei gleicher Eingabe immer gleich sein [9]. Dafür braucht es eine eindeutige Reihenfolge der

Ereignisse [10], eine idempotente Verarbeitung von Duplikaten und einen Punkt, ab dem nach einem Ausfall weitergemacht werden kann [11], [12].

### 2.3 Die Lücke

Zwischen diesen beiden Seiten gibt es eine Lücke, die in der Praxis meist anwendungsspezifisch und oft ungeprüft geschlossen wird. Es fehlt eine zusammenhängende Darstellung,

- welche Fehler in MQTT-basierten IIoT- und MES-Architekturen tatsächlich auftreten,
- welche davon von MQTT, Sparkplug B oder dem Broker abgefangen werden und welche nicht,
- und welche Architektur unter welchen Fehlern den Zustand korrekt rekonstruiert.

**Abgrenzung.** Die Arbeit betrachtet Fehlertoleranz und Zustandsrekonstruktion. IT-Sicherheit (Authentifizierung, Verschlüsselung, Angriffe) ist nicht Gegenstand der Arbeit, ebenso wenig harte Echtzeitanforderungen auf Feldebene. Andere Protokolle wie OPC UA werden nur als Quelle der Daten berücksichtigt, nicht als Alternative zu MQTT verglichen. Es wird kein neues MES entwickelt. Die Untersuchung erfolgt in einer Testumgebung und nicht im laufenden Produktivbetrieb.

## 3 Forschungsfragen und Ziele

Ziel der Arbeit ist es, den Stand der Technik zur Fehlertoleranz in MQTT-basierten Architekturen aufzuarbeiten, die relevanten Fehler in einem Fehlermodell festzuhalten und verbreitete Architekturen daran zu messen. Daraus ergeben sich drei Forschungsfragen.

**FF1** Welche Ansätze zur Fehlertoleranz und Zustandsrekonstruktion in MQTT-basierten IIoT- und MES-Architekturen beschreibt die Literatur, und welche Garantien bieten sie?

**FF2** Welche Fehler treten in solchen Architekturen auf, und welche Auswirkung haben sie auf den rekonstruierten Zustand eines MES?

**FF3** Wie gut rekonstruieren verbreitete Architekturvarianten den Zustand unter diesen Fehlern, und mit welchem Aufwand?

Die Arbeit liefert drei Ergebnisse. Erstens eine strukturierte Übersicht über den Stand der Technik und verwandte Arbeiten (FF1). Zweitens ein Fehlermodell, das Fehlerorte, Fehlerarten und ihre Auswirkung auf den Zustand beschreibt (FF2). Drittens einen Architekturvergleich mit Messergebnissen aus einer Testumgebung und einer Empfehlung für die [Firma] (FF3).

## 4 Vorgehen

### 4.1 State of the Art und Related Work

Die Literaturrecherche folgt den Leitlinien für systematische Literaturreviews nach Kitchenham und Charters [13]. Durchsucht werden IEEE Xplore, ACM Digital Library, ScienceDirect und SpringerLink. Hinzu kommen die Standards selbst (MQTT 3.1.1 und 5.0 [4], [8], Sparkplug [5], IEC 62264 [3]) sowie Dokumentationen verbreiteter Broker, da viele für die Praxis wichtige Eigenschaften nur dort beschrieben sind. Die Ergebnisse werden in vier Bereiche gegliedert:

- Zuverlässigkeit von MQTT: Dienstgüten, Sessions, Retained Messages, Broker-Clustering,
- Zustandsmodelle in der Industrie: Sparkplug B, UNS, ISA-95/IEC 62264,
- Grundlagen aus verteilten Systemen: Ordnung, Idempotenz, Checkpointing, Event Sourcing [14],

- bestehende Untersuchungen und Messungen zu Ausfällen in MQTT-Systemen.

## 4.2 Fehlermodell

Abbildung 1 zeigt die Referenzarchitektur, auf die sich das Fehlermodell bezieht. Das Fehlermodell verwendet die Begriffe Fehlerursache, Fehlerzustand und Ausfall nach Avižienis et al. [15]. Für jeden Ort in der Architektur wird festgehalten, welche Fehler dort auftreten können, wie sie sich bemerkbar machen und welchen Einfluss sie auf den Zustand im MES haben. Beispiele sind:

- Absturz und Neustart des Brokers, mit und ohne persistierte Sessions,
- Verbindungsabbruch zwischen Edge-Gateway und Broker, kurz und über längere Zeit,
- Neustart oder Überlastung eines Verbrauchers wie dem MES,
- doppelte, verspätete oder umgeordnete Nachrichten,
- Überlauf von Warteschlangen und abgelaufene Sessions,
- Netzwerkpartition in einem Broker-Cluster.

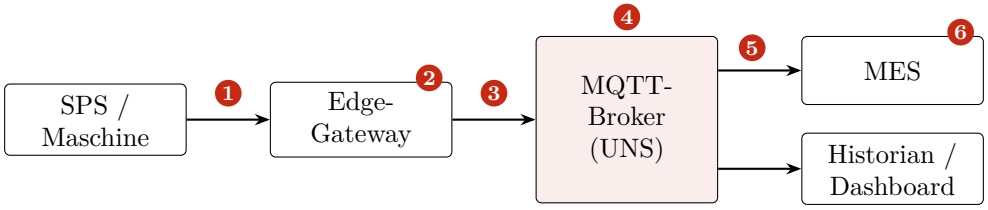



Abbildung 1: Vereinfachte Referenzarchitektur. Die Markierungen zeigen Fehlerorte: (1) Feldverbindung, (2) Edge-Gateway, (3) Verbindung zum Broker, (4) Broker bzw. Cluster, (5) Verbindung zum Verbraucher, (6) Verbraucher.

## 4.3 Architekturvergleich

Auf Basis der Literatur werden Architekturvarianten ausgewählt, die in der Praxis verbreitet sind. Die folgende Tabelle zeigt eine vorläufige Auswahl, die nach der Literaturrecherche und in Abstimmung mit der [Firma] angepasst wird.

| Variante | Prinzip | Zustand nach Fehler |
|---|---|---|
| A | Reines MQTT mit QoS 1/2, persistenten Sessions und Retained Messages | letzter Wert pro Topic, Puffer im Broker |
| B | Sparkplug B mit Birth-/Death-Nachrichten und Sequenznummern | Neuaufbau über Rebirth, Lücken werden erkannt |
| C | MQTT mit Store-and-Forward im Edge-Gateway und idempotenten Verbrauchern | Nachlieferung aus dem Edge-Puffer |
| D | MQTT mit Anbindung an ein persistentes Ereignislog (z. B. Kafka) und Event Sourcing | Wiederholung der Ereignisse ab einem Checkpoint |

Die Varianten werden zunächst qualitativ angelehnt an die Architecture Tradeoff Analysis Method (ATAM) [16] bewertet. Danach werden sie in einer containerisierten Testumgebung aufgebaut, in der eine Fertigung mit mehreren Maschinen und einem vereinfachten MES simuliert wird. Fehler aus dem Fehlermodell werden gezielt eingespielt, etwa durch Beenden von Containern oder Störung des Netzwerks, angelehnt an das Vorgehen im Chaos Engineering [17].

## 4.4 Bewertung

Für jede Variante und jeden Fehlerfall wird der rekonstruierte Zustand mit einem bekannten Sollzustand verglichen. Gemessen werden:

- **Korrektheit:** Stimmt der Zustand nach dem Fehler mit dem Sollzustand überein? Wie viele Ereignisse fehlen, sind doppelt oder in falscher Reihenfolge?
- **Determinismus:** Führt derselbe Fehler bei Wiederholung zum selben Ergebnis?
- **Wiederherstellungszeit:** Wie lange dauert es, bis der Zustand wieder korrekt ist?
- **Aufwand:** Wie hoch sind Latenz, Durchsatz, Ressourcenbedarf und Komplexität im Betrieb?

Daraus entsteht eine Zuordnung, welche Variante gegen welche Fehler schützt, und eine Empfehlung, welche Variante sich für welche Anforderungen eignet.

## 5 Nutzen für die [Firma]

Die Arbeit liefert der [Firma] eine belastbare Grundlage für Architekturentscheidungen rund um MQTT, UNS und MES. Im Einzelnen:

- ein Fehlermodell, mit dem sich bestehende und geplante Anlagen systematisch auf Schwachstellen prüfen lassen,
- einen messbasierten Vergleich von Architekturvarianten statt einer Entscheidung nach Erfahrung oder Herstellerangaben,
- eine wiederverwendbare Testumgebung, mit der sich Broker, Konfigurationen und Updates vor dem Einsatz unter Fehlerbedingungen testen lassen,
- konkrete Handlungsempfehlungen, zum Beispiel zu QoS-Einstellungen, Session-Konfiguration, Puffergrößen und Idempotenz in Verbrauchern.

Gerade weil Fehler im Datenfluss in der Fertigung oft erst spät bemerkt werden, etwa als falsche Kennzahl oder fehlende Rückmeldung, ist ein systematischer Blick auf diese Fehler auch für den Betrieb wertvoll.

## 6 Rahmenbedingungen

**Unterstützung durch das Unternehmen.** Für die Durchführung wäre Folgendes hilfreich:

- eine Ansprechperson als Betreuung im Unternehmen und regelmäßige kurze Abstimmungen,
- Einblick in die eingesetzte Architektur (Broker-Produkt, Topic-Struktur, Datenmodell, Verbraucher),
- wenn möglich anonymisierte Beispieldaten oder Logs zu bisherigen Störungen, um das Fehlermodell an realen Fällen zu prüfen,
- Zugang zu einer Test- oder Entwicklungsumgebung bzw. die Freigabe, Broker-Software in einer eigenen Umgebung zu nutzen.

**Kein Eingriff in die Produktion.** Alle Fehlerexperimente finden ausschließlich in einer isolierten Testumgebung statt. Produktivsysteme werden nicht verändert.

**Vertraulichkeit.** Unternehmensinterne Informationen werden nur in Abstimmung mit der [Firma] verwendet. Bei Bedarf wird die Arbeit mit einem Sperrvermerk versehen. Die wissenschaftlichen Ergebnisse, also Literaturübersicht, Fehlermodell und Vergleich, werden so formuliert, dass sie ohne interne Details nachvollziehbar sind. [Sperrvermerk und Veröffentlichung mit Unternehmen und Hochschule klären]

## 7 Zeitplan

Die Bearbeitungszeit beträgt sechs Monate.

| Phase | Monat | Ziel | Artefakt |
|---|---|---|---|
| 1 Einarbeitung | 1 | Bestehende Architektur bei der [Firma] verstehen, Recherche planen | Recherche-Protokoll |
| 2 State of the Art | 1–2 | Literatur und Standards auswerten | Literaturübersicht (FF1) |
| 3 Fehlermodell | 2–3 | Fehlerorte, Fehlerarten und Auswirkungen festhalten | Fehlermodell (FF2) |
| 4 Testumgebung | 3–4 | Varianten A–D und Fehlerinjektion aufbauen | Lauffähige Testumgebung |
| 5 Experimente | 4–5 | Fehlerfälle durchspielen und messen | Messergebnisse (FF3) |
| 6 Auswertung | 5 | Vergleich und Empfehlung ableiten | Bewertung, Empfehlung |
| 7 Ausarbeitung | 5–6 | Schriftliche Arbeit und Abschlusspräsentation | Masterarbeit, Code im Repository |

## Literaturverzeichnis


[1] B. Mishra und A. Kertesz, „The Use of MQTT in M2M and IoT Systems: A Survey," *IEEE Access*, Jg. 8, S. 201 071–201 086, 2020. DOI: 10.1109/ACCESS.2020.3035849.

[2] P. T. Eugster, P. A. Felber, R. Guerraoui und A.-M. Kermarrec, „The Many Faces of Publish/Subscribe," *ACM Computing Surveys*, Jg. 35, Nr. 2, S. 114–131, 2003. DOI: 10.1145/857076.857078.

[3] *IEC 62264-1:2013 – Enterprise-control system integration – Part 1: Models and terminology*, International Electrotechnical Commission, 2013.

[4] A. Banks, E. Briggs, K. Borgendale und R. Gupta, Hrsg., *MQTT Version 5.0*, OASIS Standard, OASIS, 2019. besucht am 27. Okt. 2026. Adresse: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[5] *Sparkplug Specification, Version 3.0.0*, Eclipse Foundation, 2022. besucht am 27. Okt. 2026. Adresse: https://sparkplug.eclipse.org/specification/.

[6] M. Fowler. „Event Sourcing," besucht am 27. Okt. 2026. Adresse: https://martinfowler.com/eaaDev/EventSourcing.html.

[7] J. Kreps, N. Narkhede und J. Rao, „Kafka: A Distributed Messaging System for Log Processing," in *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.

[8] A. Banks und R. Gupta, Hrsg., *MQTT Version 3.1.1*, OASIS Standard, OASIS, 2014. besucht am 27. Okt. 2026. Adresse: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html.

[9] F. B. Schneider, „Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial," *ACM Computing Surveys*, Jg. 22, Nr. 4, S. 299–319, 1990. DOI: 10.1145/98163.98167.

[10] L. Lamport, „Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Jg. 21, Nr. 7, S. 558–565, 1978. DOI: 10.1145/359545.359563.

[11] E. N. Elnozahy, L. Alvisi, Y.-M. Wang und D. B. Johnson, „A Survey of Rollback-Recovery Protocols in Message-Passing Systems," *ACM Computing Surveys*, Jg. 34, Nr. 3, S. 375–408, 2002. DOI: 10.1145/568522.568525.

[12] K. M. Chandy und L. Lamport, „Distributed Snapshots: Determining Global States of Distributed Systems," *ACM Transactions on Computer Systems*, Jg. 3, Nr. 1, S. 63–75, 1985. DOI: 10.1145/214451.214456.

[13] B. Kitchenham und S. Charters, „Guidelines for Performing Systematic Literature Reviews in Software Engineering," Keele University und Durham University, Techn. Ber. EBSE-2007-01, 2007.

[14] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[15] A. Avižienis, J.-C. Laprie, B. Randell und C. Landwehr, „Basic Concepts and Taxonomy of Dependable and Secure Computing," *IEEE Transactions on Dependable and Secure Computing*, Jg. 1, Nr. 1, S. 11–33, 2004. DOI: 10.1109/TDSC.2004.2.

[16] R. Kazman, M. Klein und P. Clements, „ATAM: Method for Architecture Evaluation," Software Engineering Institute, Carnegie Mellon University, Techn. Ber. CMU/SEI-2000-TR-004, 2000.

[17] A. Basiri u. a., „Chaos Engineering," *IEEE Software*, Jg. 33, Nr. 3, S. 35–41, 2016. DOI: 10.1109/MS.2016.60.